

FORMAN CHRISTIAN COLLEGE (A CHARTERED UNIVERSITY)



COMP 451 B

SP24

Lab 4

Hafsah Shahbaz – 251684784

Daim Bin Khalid - 251686775

INTRODUCTION

This lab report delves into the implementation of a C program designed to manipulate text files. The program aims to reinforce core concepts in C programming, particularly focusing on file handling, character manipulation, and command-line argument parsing.

The primary objective of this lab exercise is to provide hands-on experience with essential C programming techniques. By working through this lab, participants will gain a deeper understanding of file operations, character manipulation, and command-line argument handling in the C language.

Objectives:

- Implement file handling operations to read and write text files.
- Perform character manipulation to modify file contents based on specified criteria.
- Utilize command-line argument parsing to customize program behavior dynamically.

The program reads a text file, displays its contents on the console, and performs two operations: adding line numbers and removing blank lines. Upon execution, the program presents the original file's contents, followed by the same content with line numbers added and without any blank lines.

Command-line argument parsing plays a crucial role in this program, enabling users to specify input and output file paths conveniently. By parsing command-line arguments, users can define the source file, the destination file for output, and observe the desired transformations without altering the source code.

Standard C library functions serve as the backbone of this program, facilitating various tasks such as file handling, input validation, and character manipulation. These functions streamline development, ensuring efficient and reliable program execution. Below are the details of these library functions and their significance within the context of the program.

1. **<stdlib.h>** This library provides functions for memory allocation, process control, and other utility functions. In our program, it's utilized for process control.
 - `exit()`: This function terminates the program immediately, closing all open files and releasing allocated memory. It's invoked in error handling routines to gracefully exit the program in case of invalid inputs or file operations.
2. **<stdio.h>** The standard input-output library in C provides functions for file handling and console input and output operations. In our program, it's extensively used for file handling and console output.
 - `fopen()`: Opens a file specified by its filename and mode. It's utilized to open the input file in read mode for reading its content, as well as for error checking to ensure the file exists and can be accessed.
 - `fgetc()`: Reads a character from the specified file. It's used to read characters from the input file one by one.
 - `fprintf()`: Writes formatted output to a file. It's employed to write the line numbers and modified content to the output file.

- `fEOF()`: Checks for the end-of-file indicator. It's utilized to determine if the end of the input file has been reached during reading.
- `printf()`: Used to format and print output to the console. It's employed for displaying informative messages, usage instructions, and file content during program execution.
- `fclose()`: Closes the file associated with the given file pointer. This function is invoked to release system resources after reading the file content or performing decoding operations.

Each function serves a distinct purpose within the program, contributing to its overall functionality and effectiveness.

ALGORITHM and LOGIC

The program starts by checking if the correct number of command-line arguments is provided, displaying the correct usage and exiting if not. It then opens the output file in write mode, exiting with an error message if unsuccessful. The `addLineNumber()` function opens the input file in read mode and the output file in write mode, initializing a line number variable. It reads characters from the input file, adding line numbers and writing the modified content to the output file. The `removeBlankLines()` function opens the input file in read mode and the output file in write mode, initializing variables to count characters and track line numbers. It removes blank lines and specific characters from the input file, writing the modified content to the output file. The `displayFile()` function opens the specified file in read mode, displaying its content on the console until the end of file is reached. Finally, the program closes all open files.

- The program takes three command-line arguments: the input file, the file to store content with line numbers, and the file to store content without blank lines.
- The `addLineNumber()` function adds line numbers to the content of the input file and writes it to the output file specified by `inFileWithLineNo`.
- The `removeBlankLines()` function removes blank lines from the content of the input file with line numbers and writes it to the output file specified by `inFileWithoutBlanks`.
- The `displayFile()` function is used to display the content of a file on the console.

Further breakdown of code:

.addLineNumber() Function:

Functionality:

This function reads the content of the input file and adds line numbers to each line of the file. It then writes the modified content to the output file.

Arguments:

- `inFile`: Input file path.

- `inFileWithLineNo`: Output file path where the content with line numbers will be written.

Variables:

- `readFile`: File pointer for the input file.
- `lineFile`: File pointer for the output file.
- `lineNum`: Integer variable to store the line number.
- `ch`: Character variable to read each character from the input file.

Code Snippet:

```
void addLineNumber(char *inFile, char *inFileWithLineNo) {
    FILE *readFile = fopen(inFile, "r");
    FILE *lineFile = fopen(inFileWithLineNo, "w");
    int lineNum = 1;
    char ch;
    do {
        ch = fgetc(readFile);
        if (feof(readFile)) {
            break;
        }
        if (lineNum == 1) {
            fprintf(lineFile, "%d. ", lineNum);
            lineNum++;
        }
        if (ch == '\n') {
            fprintf(lineFile, "\n%d. ", lineNum);
            lineNum += 1;
        } else {
            fprintf(lineFile, "%c", ch);
        }
    } while (1);
    fclose(readFile);
    fclose(lineFile);
}
```

Explanation:

- The function opens the input file in read mode and the output file in write mode.
- It initializes `lineNum` to 1 and reads characters from the input file one by one.
- For each character read, if it's a newline character, the function writes the current line number followed by a dot and space to the output file and increments `lineNum`.
- Finally, it closes both input and output files.

2. `removeBlankLines()` Function:

Functionality:

This function removes blank lines from the content of the input file and writes the modified content to the output file.

Arguments:

- `inFileWithLineNo`: Input file path containing content with line numbers.
- `inFileWithoutBlanks`: Output file path where the content without blank lines will be written.

Variables:

- `readFile`: File pointer for the input file with line numbers.
- `blankFile`: File pointer for the output file without blank lines.
- `ch`: Character variable to read each character from the input file.
- `charCount`: Integer variable to count characters in each line.
- `lineNum`: Character variable to store the current line number.

Code Snippet:

```
void removeBlankLines(char *inFileWithLineNo, char *inFileWithoutBlanks) {
    FILE *readFile = fopen(inFileWithLineNo, "r");
    FILE *blankFile = fopen(inFileWithoutBlanks, "w");
    char ch;
    int charCount = 0;
    char lineNum = '1';
    do {
        ch = fgetc(readFile);
        charCount++;
        if (feof(readFile)) {
            break;
        }
        if (charCount == 4 && ch == '\n') {
            charCount = 3;
            fseek(readFile, 3, SEEK_CUR);
            continue;
        }
        if (ch == '\n') {
            charCount = 0;
        }

        fprintf(blankFile, "%c", ch);
    } while (1);
    fclose(readFile);
    fclose(blankFile);
}
```

Explanation:

- The function opens the input file (with line numbers) in read mode and the output file in write mode.
- It reads characters from the input file one by one and removes blank lines based on the character count in each line.
- Finally, it closes both input and output files.

3. displayFile () Function:

Functionality:

This function displays the content of a file on the console.

Arguments:

- `filename`: File path of the file to be displayed.

Variables:

- `fp`: File pointer for the file.

Code Snippet:

```
void displayFile(char *filename) {  
    FILE *fp = fopen(filename, "r");  
    char ch;  
    do {  
        ch = fgetc(fp);  
        printf("%c", ch);  
    } while (ch != EOF);  
    fclose(fp);  
}
```

Explanation:

- The function opens the file specified by `filename` in read mode.
- It reads characters from the file one by one and prints them to the console until the end of file (EOF) is reached.
- Finally, it closes the file.

These functions collectively provide the functionality to add line numbers, remove blank lines, and display the content of files, contributing to the overall functionality of the program.

main () Function:

Functionality:

The `main()` function is the entry point of the program. It orchestrates the execution of other functions by processing command-line arguments, opening files, invoking functions to modify file content, and displaying results on the console.

Arguments:

- `argc`: An integer representing the number of command-line arguments passed to the program.
- `argv`: An array of strings containing the command-line arguments.

Variables:

- `fp3`: File pointer for the output file.
- `argc`: Integer variable storing the number of command-line arguments.
- `argv`: Array of strings storing the command-line arguments.

Code Snippet:

```
int main(int argc, char *argv[]) {
    if (argc != 4) {
        printf("Usage: %s <inFile> <inFileWithLineNo>
<inFileWithoutBlanks>\n", argv[0]);
        return 1;
    }
    FILE *fp3 = fopen(argv[3], "w");
    if (fp3 == NULL) {
        printf("Couldn't open %s\n", argv[3]);
        return 1;
    }
    addLineNumber(argv[1], argv[2]);
    removeBlankLines(argv[2], argv[3]);
    printf("\n====ORIGINAL FILE====\n");
    displayFile(argv[1]);
    printf("\n\n====FILE WITH LINE NUMBER====\n");
    displayFile(argv[2]);
    printf("\n\n====FILE WITHOUT BLANK LINES====\n");
    displayFile(argv[3]);
    return 0;
}
```

Explanation:

- The function begins by checking if the correct number of command-line arguments (four) is provided. If not, it prints the correct usage and exits.
- It opens the output file specified by `argv[3]` in write mode. If unsuccessful, it prints an error message and exits.
- The `addLineNumber()` function is called to add line numbers to the content of the input file (`argv[1]`) and write it to the output file (`argv[2]`).
- The `removeBlankLines()` function is called to remove blank lines from the content of the file with line numbers (`argv[2]`) and write it to the output file without blank lines (`argv[3]`).
- It then displays the original file, the file with line numbers, and the file without blank lines on the console using the `displayFile()` function.
- Finally, the function returns 0, indicating successful execution.

Overall, the `main()` function manages the execution flow of the program, ensuring proper file handling, content modification, and output display.

OUTPUT

Input:

- Input file (input.txt):

Hello friends!

This is compiler construction lab

C programming is real fun.

Have a nice day.

Scenario 1:

```
• hafsah on Hafsah at ~/lab4 ./task input.txt lines.txt blanks.txt

====ORIGINAL FILE====
Hello friends!

This is compiler construction lab

C programming is real fun.
Have a nice day.💎

====FILE WITH LINE NUMBER====
1. Hello friends!
2.
3.
4. This is compiler construction lab
5.
6.
7. C programming is real fun.
8. Have a nice day.💎

====FILE WITHOUT BLANK LINES====
1. Hello friends!
2. This is compiler construction lab
3. C programming is real fun.
4. Have a nice day.💎
```

Comments:

- The input file contains four lines, including two blank lines.
- The program adds line numbers to each line in the lines.txt file.
- It removes the blank lines and retains only the lines with content in the blanks.txt file.

Scenario 2:

```
hafsah on Hafsah at ../lab4 ./task input.txt
Usage: ./task <inFile> <inFileWithLineNo> <inFileWithoutBlanks>

hafsah on Hafsah at ../lab4
```

- The command is missing two required command-line arguments (<inFileWithLineNo> and <inFileWithoutBlanks>).
- The program displays a usage message indicating the correct usage of the program, which requires three command-line arguments.

Scenario 3:

```
hafsah on Hafsah at ../lab4 ./task file1 file2 file3
Couldn't open file1

hafsah on Hafsah at ../lab4
```

- The program attempts to open the file specified by `file1` but fails to do so.
- It prints an error message indicating that it couldn't open `file1` since it doesn't exist.